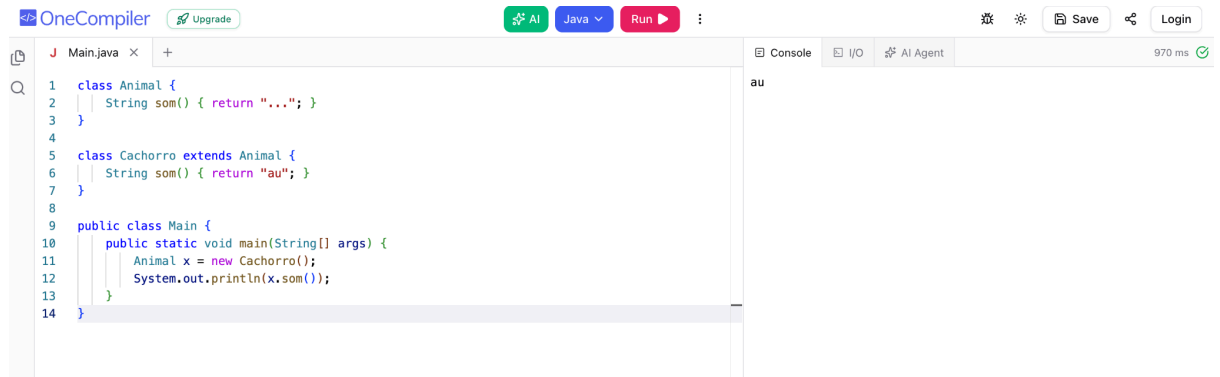


Atividade aula-009 - Paradigmas e Linguagens

Heitor Henrique Scramim de Freitas - RA: 24190382-2

1. a)

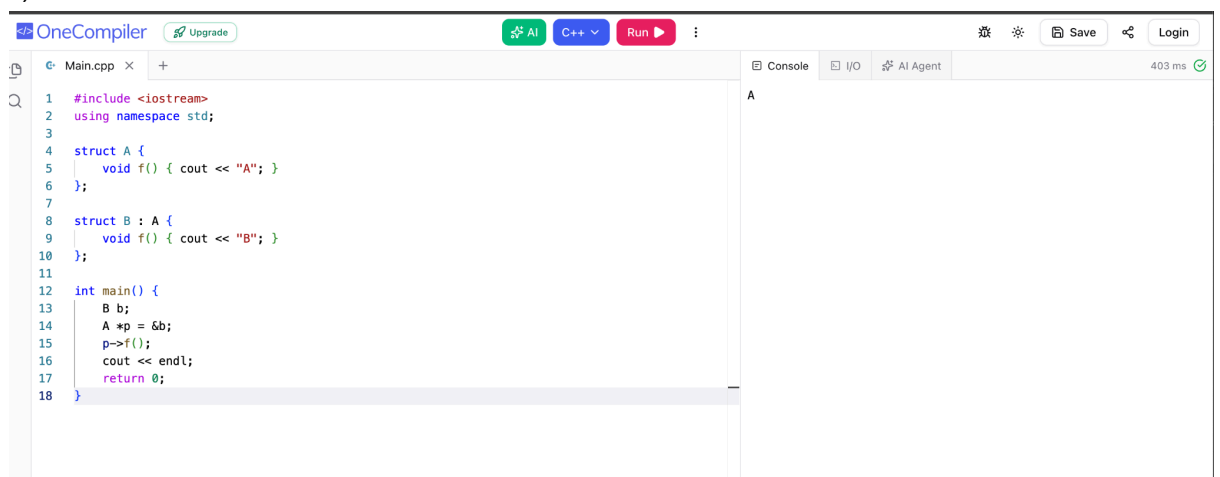


```
1 class Animal {
2     String som() { return "..."; }
3 }
4
5 class Cachorro extends Animal {
6     String som() { return "au"; }
7 }
8
9 public class Main {
10     public static void main(String[] args) {
11         Animal x = new Cachorro();
12         System.out.println(x.som());
13     }
14 }
```

The console output shows "au", indicating that the JVM resolved the call to the overridden method in the actual object type (Cachorro) at runtime.

b) Execução. Mesmo com a variável declarada como Animal, o objeto criado é um Cachorro. A JVM olha para o objeto real na hora da chamada e executa o som() dele. É vinculação dinâmica.

2. a)



```
1 #include <iostream>
2 using namespace std;
3
4 struct A {
5     void f() { cout << "A"; }
6 };
7
8 struct B : A {
9     void f() { cout << "B"; }
10 };
11
12 int main() {
13     B b;
14     A *p = &b;
15     p->f();
16     cout << endl;
17     return 0;
18 }
```

The console output shows "A", indicating that the compiler resolved the call to A::f() at compile time because f() was not virtual.

b) Compilação. Como f não foi declarada virtual, o compilador resolve a chamada só pelo tipo do ponteiro (A*) e já fixa A::f. Se A tivesse virtual void f(), a saída seria B.

3. a)

```
1 class A {
2     String nome = "A";
3     String getNome() { return nome; }
4 }
5
6 class B extends A {
7     String nome = "B";
8     String getNome() { return nome; }
9 }
10
11 public class Main {
12     public static void main(String[] args) {
13         A x = new B();
14         System.out.println(x.nome + " " + x.getNome());
15     }
16 }
```

Console: A B

b) Compilação para o campo, execução para o método. O `x.nome` usa o tipo declarado da variável (A), porque campos não são polimórficos e o nome de B apenas esconde o de A. Já o `x.getNome()` é despachado dinamicamente para a versão de B, que devolve o nome de B. Por isso é boa prática manter os campos `private`.

4. a)

```
1 class Contador:
2     total = 0
3
4     def __init__(self):
5         Contador.total += 1
6         self.id = Contador.total
7
8
9 a = Contador()
10 b = Contador()
11 print(a.id, b.id, a.total)
```

Console: 1 2 2

b) Execução. Python resolve os nomes ao rodar. O `total` é uma variável de classe, com uma única cópia compartilhada, e chega a 2 depois dos dois objetos. O `id` é de instância, cada objeto guarda o seu (1 e 2). O `a.total` não existe em `a`, então o Python busca na classe e encontra 2.

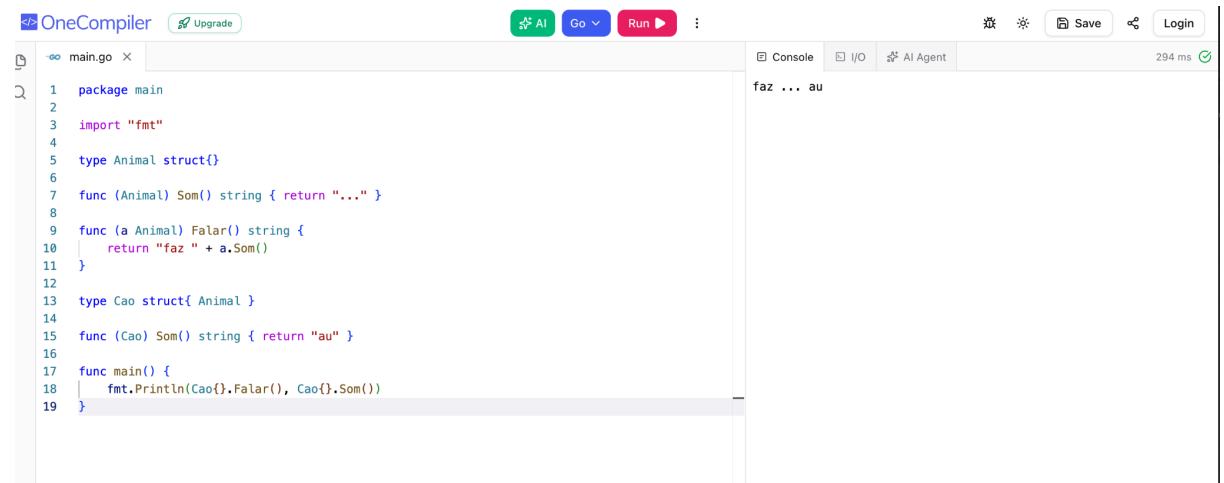
5. a)

```
1 class A {
2     static String quem() { return "A"; }
3 }
4
5 class B extends A {
6     static String quem() { return "B"; }
7 }
8
9 public class Main {
10     public static void main(String[] args) {
11         A x = new B();
12         System.out.println(x.quem());
13     }
14 }
```

Console: A

b) Compilação. Método static não é sobrescrito, é escondido. A chamada é resolvida pelo tipo declarado da variável (A), então roda o quem() de A. O compilador ou a IDE até avisa que o certo seria chamar A.quem()).

6. a)



```
1 package main
2
3 import "fmt"
4
5 type Animal struct{
6
7 func (Animal) Som() string { return "..."}
8
9 func (a Animal) Falar() string {
10     return "faz " + a.Som()
11 }
12
13 type Cao struct{ Animal }
14
15 func (Cao) Som() string { return "au" }
16
17 func main() {
18     fmt.Println(Cao{}.Falar(), Cao{}.Som())
19 }
```

Console: faz ... au 294 ms

b) Compilação. Embedding em Go não é herança. Dentro de Animal.Falar, a.Som() sempre chama o Som de Animal, porque não há despacho dinâmico entre o struct embutido e o que o contém. Cao{}.Som() chama o Som do próprio Cao. Para ter polimorfismo em Go é preciso usar uma interface.